

From specification to bug evidence

A 32-bit BMU.

A complete verification story.

SystemVerilog + UVM

Directed + random testing

Assertions + coverage

Original RTL. Reproducible failures.

17

operation classes

562

functional bins hit

9

open DUT findings

Follow one MAX request from sequence creation to root-cause evidence.

The route through the project

Click a section to jump there. Use MENU on any slide to return.

01 Understand

Specification, controls, timing,
DUT

02 Follow a request

Environment and MAX
transaction

03 Test the design

Directed, random, assertions

04 Measure + debug

Coverage, results, nine
findings

05 Correct + close

Existing fix code and
validation gate

06 Explore details

Operations, evidence,
commands

Audience checkpoints: predict a result • classify a failure • interpret coverage.

What does this BMU do?

32-bit operands in; a registered result and a live error out.

Logic and shift

OR, XOR, SRL, SRA, ROR, BINV

Arithmetic and compare

SH2ADD, SUB, SLT/SLTU, signed MAX

Count and rearrange

CTZ, CPOP, SEXT.B, PACK, GREV

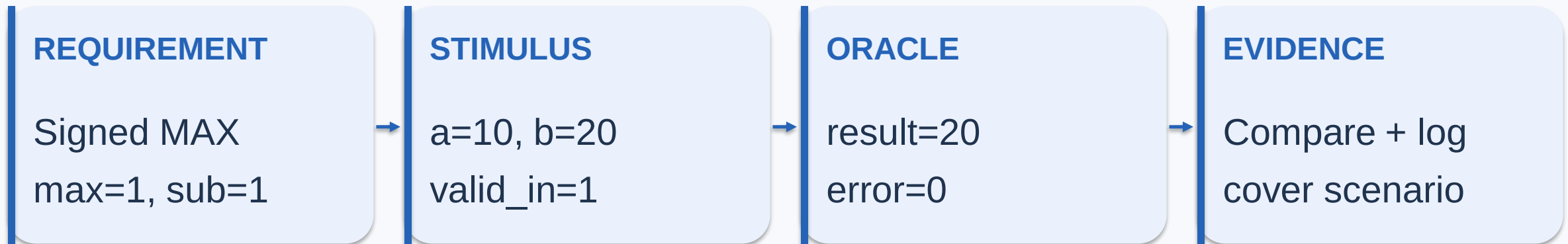
CSR data paths

Bypass read and write-data selection

Predict: CPOP(0xFFFF0000) = ? Keep your answer for the bug review.

Turn the specification into checks

Example requirement: signed MAX selects the larger operand and requires SUB.



Negative companion test

Remove SUB: require error=1 and captured result=0.

Record ambiguities

Timing, GREV invalid encoding, CSR conflicts.

One selected operation can require several control bits. “Legal” is not simply one-hot.

The interface contract determines sampling

No ready signal. No output-valid handshake. `valid_in` enables result capture.

Inputs

`a_in`, `b_in`: 32 bits
`ap`: 42 control bits
CSR enable + read data
`rst_l`, `valid_in`,
`scan_mode`

Falling edge

Driver applies request

Between edges

Combinational paths settle

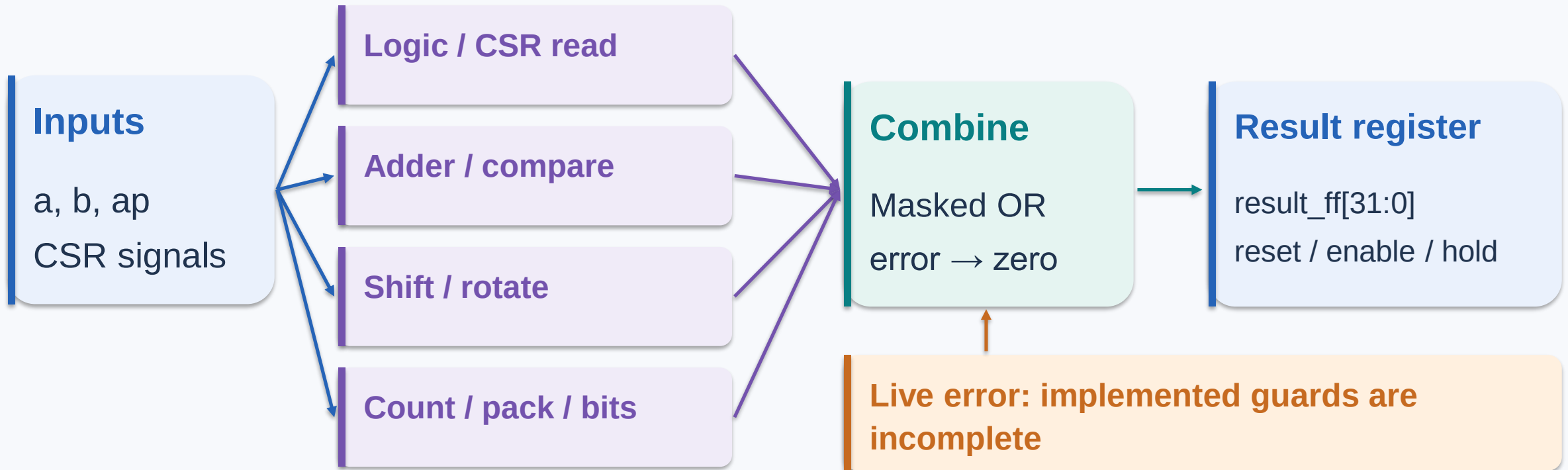
Rising edge

Capture result; monitor samples

`valid_in=0` → result holds. Invalid controls can still assert the live error output.

Inside the DUT

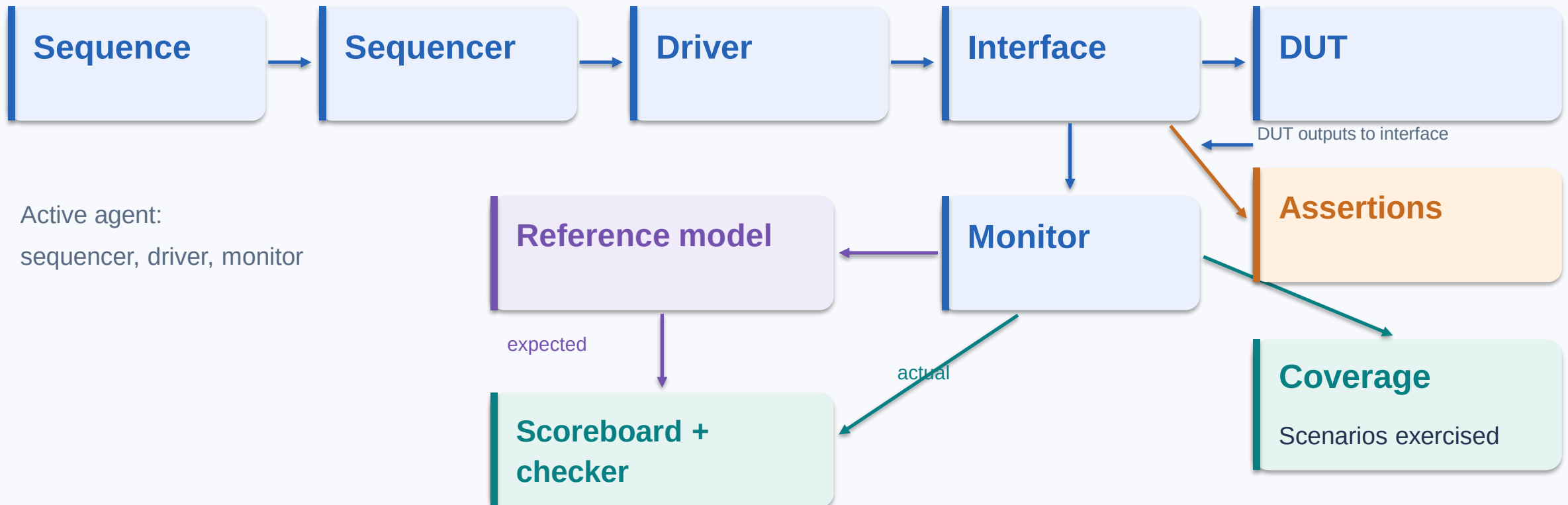
Parallel datapaths feed a combined result; implemented error logic gates capture data.



Data bugs and missing control guards need different checks.

The UVM environment: two flows, one verdict

Stimulus travels toward the DUT. Observations fan out to checking and coverage.



Predict from observed inputs. Compare both result_ff and error.

Your turn: construct a legal MAX request

Before revealing the DUT output, predict what should happen.

```
// Transaction values
```

```
rst_l      = 1;
```

```
valid_in   = 1;
```

```
ap.max     = 1;
```

```
ap.sub     = 1;
```

```
a_in       = 10;
```

```
b_in       = 20;
```

Question 1

Is max=1 alone a legal request?

Question 2

What are result_ff and error?

Think first. The next two slides reveal when the request is driven and what happens.

The request becomes pin-level activity

Sequence → sequencer → driver → interface → DUT

```
// UVM handshake – simplified
start_item(req);
// fill request fields
finish_item(req);

// Driver: get → drive → item_done
```

Driver completion means the item was driven.
The scoreboard determines correctness.

10 ns: falling edge

Drive MAX, SUB, a=10, b=20

10–15 ns: compute

Result path and live error settle

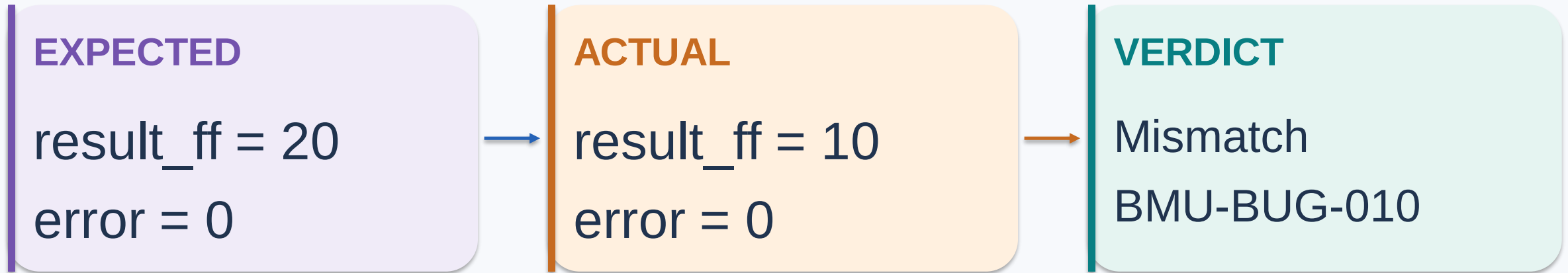
15 ns: rising edge

Register updates; monitor samples

Expected after capture: result_ff=20, error=0. The next slide reveals the actual result.

The scoreboard exposes the MAX defect

Same observed request. Independent prediction. Different data.



```
@ 15 ns    expected: 0x00000014 / error=0    actual: 0x0000000a / error=0
```

A scenario can be covered and incorrect at the same time.

Who checks the checker?

A useful failure must distinguish a DUT defect from a testbench mistake.

865 predictor self-checks

Legal, invalid, boundary and state cases

Passing setup checks

OR smoke and timing/reset tests

Complete comparisons

Result + error; four-state equality; queue checks

Honest failure detection

UVM errors, assertions and incomplete runs fail

Prediction is independent of RTL. Coverage shares specification classification helpers.

Directed testing asks precise questions

Select inputs because they target a requirement, a boundary, or a suspected defect.

Type	Example	Expected check
Nominal	OR with known a and b	Exact result and error=0
Boundary	CTZ(0); shift by 0 or 31	Defined edge behavior
Negative control	MAX without SUB	error=1; capture zero
Timing / reset	valid_in=0; reset mid-traffic	Hold, clear, recover
Bug reproducer	MAX(10,20)	Detect the isolated defect

Small, intentional tests explain failures quickly.

Three kinds of random exploration

Each family generates 100 requests per run; the monitored reset adds one sample.

Legal random

Valid operation + legal modifiers

Broad operand combinations

Seeds 101 / 102 / 103

Corner random

Independent corners for a and b

0, all ones, signed limits, patterns

Seeds 201 / 202 / 203

Error random

Ten invalid-request classes

Conflicts and missing requirements

Seeds 301 / 302 / 303

Directed tests target known risks. Random tests explore combinations. Seeds reproduce stimulus.

Assertions guard behavior across cycles

The scoreboard checks data. Assertions express signal-level and temporal rules.

```
@(posedge clk) disable iff (!rst_1)
    !valid_in | => $stable(result_ff);
```

Reset

- Clear result
- Suppress error

Timing / hold

- Hold when disabled
- Known enabled result

Control guards

- CSR conflict
- Missing SUB / ZBA

Audience check: result holds, but error changes while valid_in=0. Is that allowed?

Coverage measures exercise, not correctness

Different coverage types answer different questions.

Functional coverage

Did we exercise the planned scenarios?
Operations, modes, count values, crosses

Code coverage

Which RTL structures were exercised?
Blocks, scored expressions, toggles

Assertion coverage

Which properties were exercised?
Read coverage separately from failures

Can functional coverage be 100% while the DUT is wrong? Yes—this project demonstrates it.

Beyond smoke tests: systematic stress

Finite sweeps complement random testing and make coverage closure deliberate.

Control mutation matrix

Legal base → add forbidden field
Also remove required SUB / ZBA

Boundary enumeration

32 shift/index values; counts 0–32
256 bytes; 32 GREV encodings

Race-aware checking

Falling-edge drive, post-update sample
Reset/hold state in the predictor

Reproducible evidence

Effective seeds, isolated logs
Source and log SHA-256 fingerprints

Coverage closure uses 5,036 comparisons on the original DUT; failures remain visible.

Measured results: exercise achieved, bugs remain

Recorded baseline: 22 September 2026 • original DUT • coverage test seed 4

26

runs: 3 pass / 23 fail

562 / 562

functional bins hit

9

open DUT findings

Functional



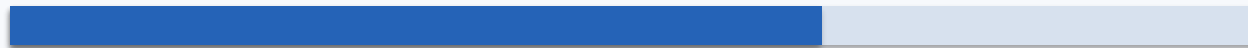
100.00%

DUT blocks



16 / 16

DUT toggles



1,230 / 1,892

3/3 scored expressions only. Toggle coverage is incomplete. No DUT correctness sign-off.

Nine findings, four recurring defect patterns

All open; one critical and eight major. Each has a seed-1 isolated reproducer.

Count / bit ordering

001: CPOP ignores upper half
009: CTZ reversal ordering

Data arrangement

002: PACK halves reversed
005: GREV swaps halfwords

Source selection

003: CSR sources reversed
010: MAX chooses wrong operand

Missing rejection

006: incomplete guards
007: missing SUB; 008: invalid GREV

BUG-008 depends on CLARIF-004. BUG-004 was withdrawn; pure CSR bypass is legal.

MAX root cause: the selector is inverted

The isolated request removes unrelated controls and makes the cause visible.

```
bitmanip_minmax_sel_a = ge ^ ap_max;    // original RTL
```

Compare

10 >= 20
ge = 0



Select

0 XOR 1
select_a = 1



Observe

a is selected
result = 10

Reproduce → isolate → inspect cause → propose correction → rerun related tests.

What the exploratory RTL correction changes

The repository contains fix_v1 code. It is separate from the measured submission DUT.

```
// MAX: select using a signed comparison
corrected_result = ($signed(a_in) > $signed(b_in)) ? a_in : b_in;
```

Path	Correction present in fix_v1
CPOP / CTZ	Count all 32 bits; scan CTZ from bit 0, default 32
PACK / GREV	Restore halfword order / reverse all four bytes
CSR write	csr_imm ? b_in : a_in
Control checks	Separate guard logic and required-control checks

Code changes exist; corrected-DUT regression closure is not established by this evidence.

When can we honestly call a bug fixed?

A code edit is the start of closure, not the end.

1 Reproduce

Original DUT fails
Same test and seed

2 Retest

Corrected DUT
passes
Same expected
result

3 Expand

Related boundaries
Full regression

4 Record

Source + seed +
logs
Update bug
disposition

Current status: nine original-DUT bugs open; fix_v1 remains exploratory.

What this verification established

A traceable path from requirements to reproducible evidence.

Established

17 operation classes exercised
865 predictor self-checks passed
562 functional bins exercised
Nine reproducible DUT findings

Still open

Original RTL defects
Corrected-DUT validation
Incomplete toggle coverage
Three documented assumptions

The deliverable is reproducible verification evidence, with clear limits and open findings.

Operation reference

32-bit results; shift/index uses b[4:0]; arithmetic wraps modulo 2^{32} .

Family	Expected behavior / legal mode
OR / XOR	Normal b, or inverted b with zbb=1
SRL / SRA / ROR / BINV	Logical / arithmetic / rotate / invert indexed bit
SH2ADD / SUB	(a<<2)+b with ZBA; a-b without ZBA
SLT / MAX	Require SUB; SLT allows unsigned; MAX is signed
CTZ / CPOP / SEXT.B	Trailing zeros (zero→32); set bits; sign-extend bit 7
PACK / GREV	{b[15:0],a[15:0]}; byte reverse for b[4:0]=24
CSR read / write	Pure bypass; write selects b if immediate else a

Bug evidence: data-path defects

All examples reproduced on the original DUT, seed 1. Hex values have a 0x prefix.

Bug	Stimulus	Expected → observed
001 CPOP	a=0xFFFF0000	16 → 0
002 PACK	a=0x1234, b=0x5678	0x56781234 → 0x12345678
003 CSR	a=0x33334444, b=0x11112222	Immediate and register sources reversed
005 GREV	a=0x12345678, b=24	0x78563412 → 0x56781234
009 CTZ	a=1	0 → 1
010 MAX	a=10, b=20; max+sub	20 → 10

Bug evidence: control guards and assumptions

Invalid captured requests require zero result and error=1 under the adopted rules.

Bug	Invalid request	Failure
006 • Critical	Empty ap; no CSR read	error=0 instead of 1
007 • Major	SLT or MAX without SUB	error=0; MAX may return nonzero data
008 • Major	GREV with b[4:0]=5	error=0 under CLARIF-004

Recorded assumptions

001: timing/reset/scan • 004: invalid GREV • 005: pure CSR read versus conflict

Assertion and coverage reference

Use properties, functional bins, and code metrics together.

```
reset_suppresses_error  
reset_clears_result  
result_holds_when_invalid  
valid_result_is_registered  
one_primary_operation  
empty_valid_request  
live_error_when_invalid  
csr_conflict  
sh2add_requires_zba  
sub_rejects_zba  
slt_requires_sub  
max_requires_sub  
grev_encoding
```

Functional dimensions

Operations × patterns; valid/error
Shift/index; CTZ/CPOP count; modes

Measurement limits

11/13 assertion coverage ≠ pass rate
3 scored expressions ≠ every expression
65.01% toggles; no FSM claim

Reproduce the evidence

Run from the repository root with Xcelium and a working Cadence license.

```
# Passing setup checks
make -C sim run TEST=bmu_model_self_test SEED=1
make -C sim run TEST=bmu_timing_reset_test SEED=1

# Full regression and isolated MAX failure
make -C sim regression
make -C sim run TEST=bmu_bug_010_test SEED=1

# Coverage extraction after the coverage test
imc -exec sim/scripts/report_coverage.tcl
```

Expected DUT failures remain failures. Zero UVM fatals does not mean every test passed.

Where to find the supporting evidence

The README is the project walkthrough; these files hold the detailed records.

Question	Repository artifact
What was required?	docs/00_spec/BMU_Specification_v1.2.pdf
What was tested?	docs/02_test_plan/BMU_Test_Plan.md
What failed?	docs/04_bug_reports/BMU_Bug_Log.md + results/bugs/
What ran?	results/reports/regression_summary.csv
What produced the results?	results/reports/run_manifest.json
What are the limits?	docs/08_signoff/BMU_Signoff_Report.md

Questions? Return to MENU to revisit any part of the verification story.